Laporan Praktikum 7.5.1

Langkah Kerja

1. Membuat direktori kerja

Buat folder iot-server3, lalu di dalamnya buat sub-direktori mosquitto/config/.

```
D:\SEMESTER 3\IOT-SERVER3
└──mosquitto
```

2. Membuat mosquitto.conf

Simpan di iot-server3/mosquitto/config/mosquitto.conf:

mosquitto > config > mosquitto.conf

```
# ================================================================================
# KONFIGURASI DASAR ECLIPSE MOSQUITTO BROKER
# ================================================================================

# Listener Saluran MQTT Standar (Untuk NodeMCU ESP8266 & Klien TCP)
listener 1883
protocol mqtt

# Listener Saluran WebSockets (Untuk Aplikasi Web Browser & Dashboard Visual)
listener 9001
protocol websockets

# Perizinan Akses Awal (Tanpa Password)
allow_anonymous true

# Konfigurasi Persistensi Penyimpanan Pesan
persistence true
persistence_location /mosquitto/data/

# Konfigurasi Berkas Log
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

3. Membuat docker-compose.yml

Simpan di folder iot-server/:

docker-compose.yaml

```yaml
services:
  # ============================================================
  # 1. Layanan Broker MQTT: Eclipse Mosquitto
  # ============================================================
  mqtt:
    image: eclipse-mosquitto:latest
    container_name: mqtt-broker
    restart: unless-stopped
    ports:
      - "1883:1883"  # Port MQTT murni untuk mikrokontroler
      - "9001:9001"  # Port WebSockets untuk dashboard web
    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log

  # ============================================================
  # 2. Layanan Visual Rule Engine: Node-RED
  # ============================================================
  nodered:
    image: nodered/node-red:latest
    container_name: node-red
    restart: unless-stopped
    ports:
      - "1880:1880"  # Port antarmuka web builder Node-RED
    environment:
      - TZ=Asia/Jakarta
    volumes:
      - ./nodered:/data
    depends_on:
      - mqtt
```

4. Masuk ke direktori iot-server3 lewat terminal dan menjalankan container

```
docker compose up -d
```

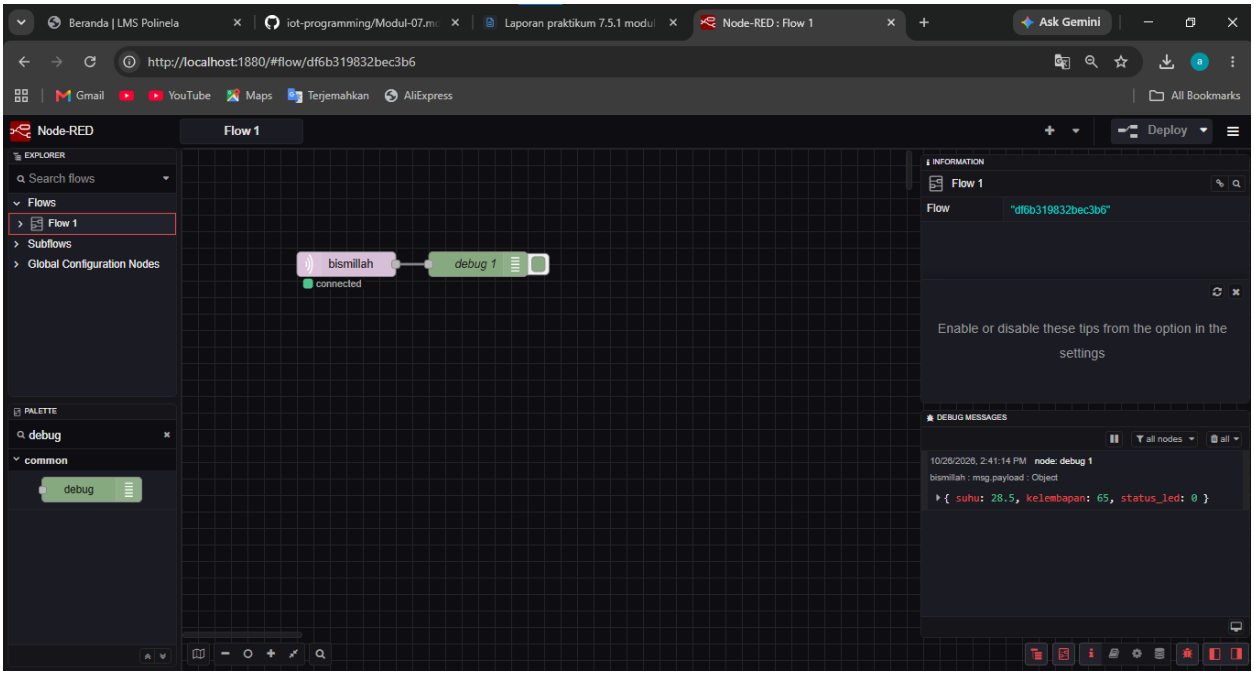
Tunggu sampai muncul [+] Running 2/2 dengan status Started. Jika memakai Docker versi lama, gunakan docker-compose up -d.

```
D:\Semester 3\iot-server3>docker compose up -d
[+] up 23/25
 ⠿ Image nodered/node-red:latest  Pulled                                                                282.7s
 ⠿ Image eclipse-mosquitto:latest Pulled                                                                 63.1s
 ⠿ Network iot-server3_default    Created                                                                 0.7s
 ⠿ Container mqtt-broker          Started                                                                16.2s
 ⠿ Container node-red             Started                                                                10.1s

D:\Semester 3\iot-server3>docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED         STATUS                   PORTS
                                                                                 NAMES
5358db0ec864   nodered/node-red:latest  "./entrypoint.sh"        5 minutes ago   Up 5 minutes (healthy)   0.0.0.0:1880
->1880/tcp, [::]:1880->1880/tcp                                                  node-red
298d9eb51c2e   eclipse-mosquitto:latest "/docker-entrypoint.…"   5 minutes ago   Up 5 minutes             0.0.0.0:1883
->1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp   mqtt-broker
```

5. Mengakses Node-RED

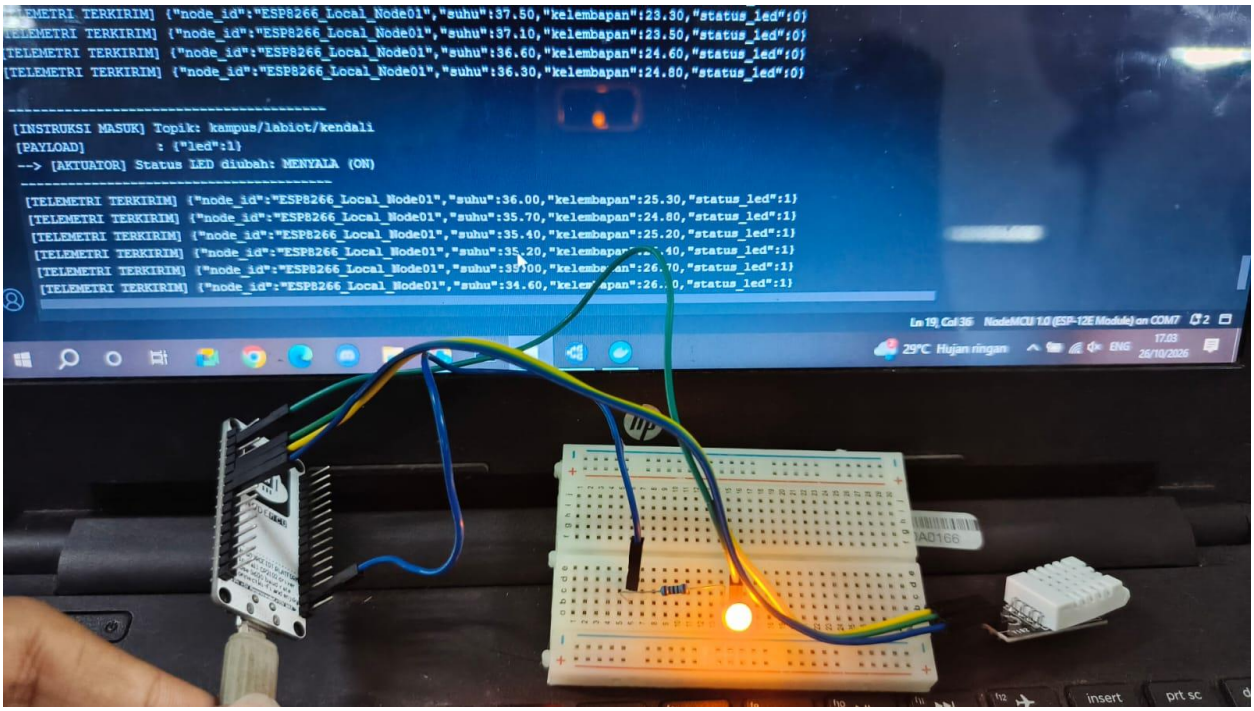
Buka browser lalu akses http://localhost:1880. Jika kanvas Node-RED tampil, server IoT lokal berhasil berjalan.



7.5.2

Langkah Kerja

1. **Merakit rangkaian.** DHT: + ke 3V3, OUT ke D4 (GPIO 2), – ke GND. LED: anoda ke D1 (GPIO 5), katoda lewat resistor 220 Ω ke GND. Screenshot/foto rangkaian.



2. **Mengecek IPv4 laptop.** Jalankan `ipconfig` di cmd, catat IPv4 dari adapter Wi-Fi

```
Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::51d8:de0f:acc3:d1f9%3
   IPv4 Address. . . . . . . . . . . : 192.168.181.108
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.181.228
```

3. **Menulis kode program.** Salin kode dari modul, lalu ganti `ssid`, `password`, dan `mqttServer` (IP laptop dari langkah 2). Screenshot bagian konfigurasi kode.

```
// 1. Kredensial Wi-Fi (Pastikan Laptop dan ESP8266 berada di Jaringan yang Sama!)
const char* ssid     = "trrrr";
const char* password = "12345678";

// 2. Konfigurasi Broker MQTT Lokal
// Ganti dengan Alamat IPv4 Laptop Anda yang didapat dari perintah 'ipconfig'!
const char* mqttServer = "192.168.181.108";
const int   mqttPort   = 1883;

// 3. Konfigurasi Identitas Simpul & Topik
const char* clientId      = "ESP8266_Local_Node01";
const char* topicPubTele  = "kampus/labiot/telemetri";
const char* topicSubCmd   = "kampus/labiot/kendali";

// 4. Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)

#define DHTTYPE DHT22 // Ganti DHT22 jika modul sensor berwarna putih

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);
```

4. **Membuka Serial Monitor (115200 baud).** Pastikan muncul Berhasil Terhubung!, IP NodeMCU, Terhubung!, dan Berlangganan ke topik dan emastikan telemetri terkirim. Pastikan muncul baris [TELEMETRI TERKIRIM] {...} dengan nilai suhu dan kelembapan.

```
[Wi-Fi] Menghubungkan ke trrrr
.....
[Wi-Fi] Berhasil Terhubung!
[Wi-Fi] IP NodeMCU: 192.168.181.219
[MQTT] Menghubungkan ke Broker Lokal (192.168.181.108)... Terhubung!
[MQTT] Berlangganan ke topik: kampus/labiot/kendali
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":38.10,"kelembapan":22.20,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":37.80,"kelembapan":22.80,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":37.50,"kelembapan":23.30,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":37.10,"kelembapan":23.50,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":36.60,"kelembapan":24.60,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":36.30,"kelembapan":24.80,"status_led":0}

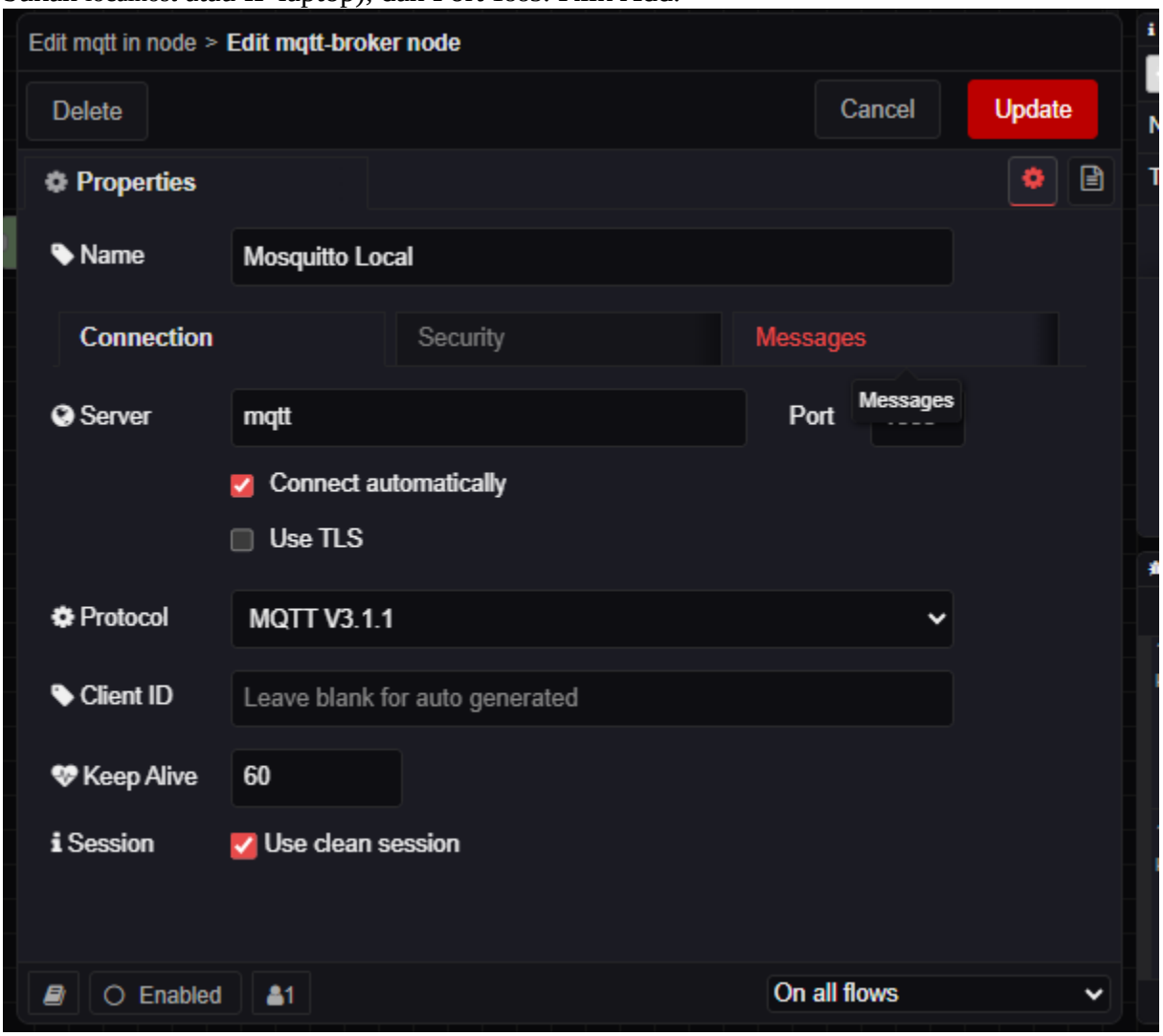
----------------------------------------
[INSTRUKSI MASUK] Topik: kampus/labiot/kendali
[PAYLOAD]         : {"led":1}
--> [AKTUATOR] Status LED diubah: MENYALA (ON)
----------------------------------------
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_Node01","suhu":36.00,"kelembapan":25.30,"status_led":1}
```

7.5.3

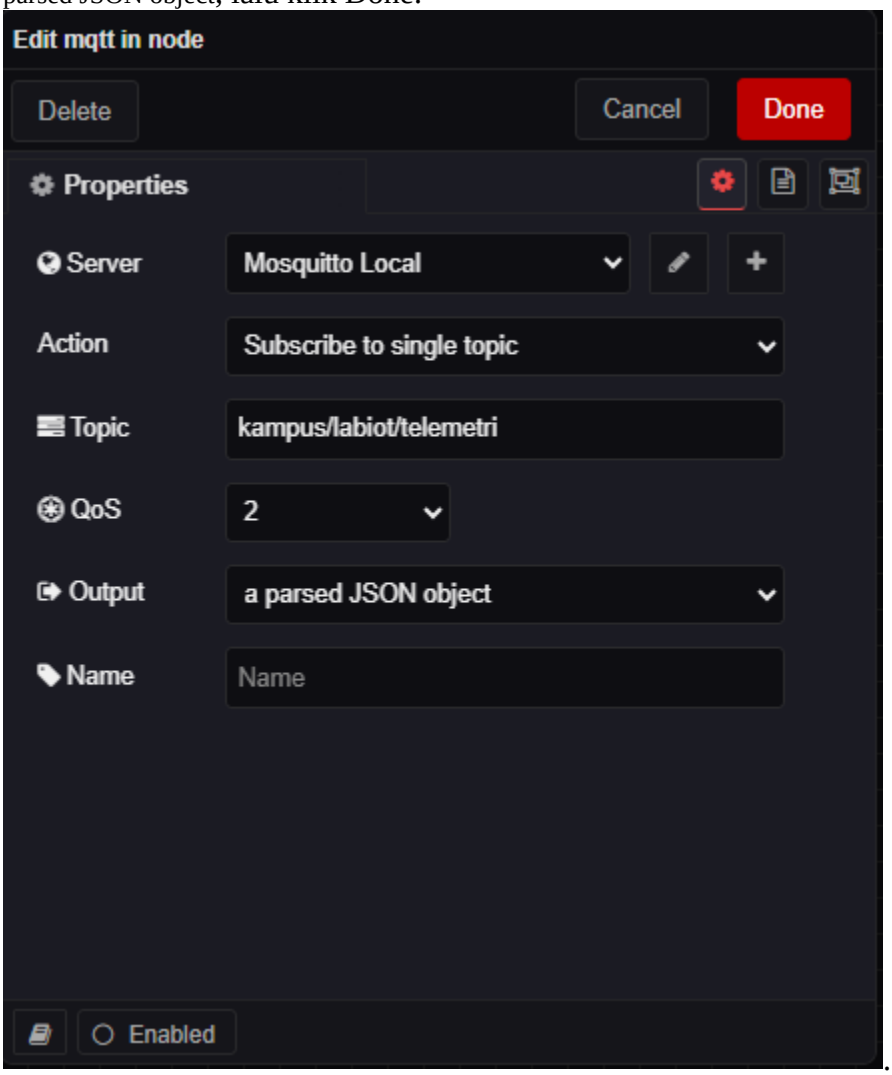
Langkah Kerja

1. Membuka Node-RED di http://localhost:1880 .Menambahkan node mqtt in. Seret dari palette kategori network ke kanvas.

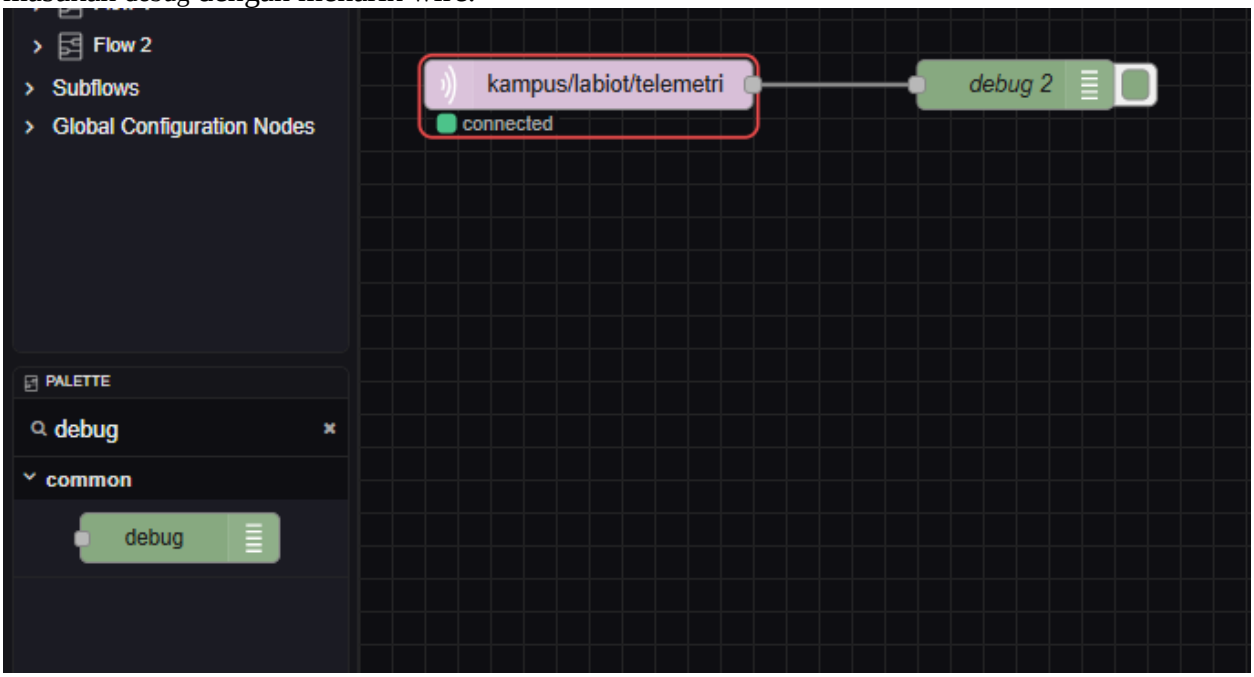
2. Membuat konfigurasi server broker. Klik dua kali node mqtt in, klik ikon pensil pada baris Server, lalu isi Name Mosquitto Local, Server mqtt (nama layanan di Docker Compose, bukan localhost atau IP laptop), dan Port 1883. Klik Add.



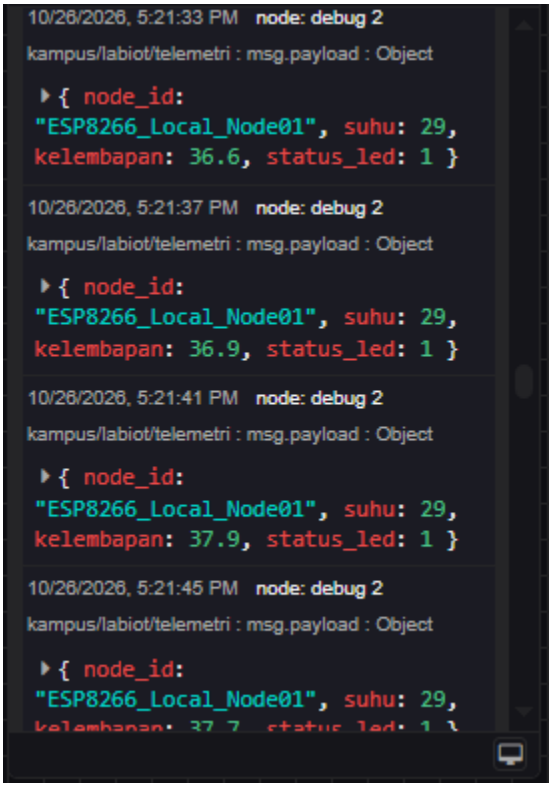
3. Mengatur topik dan format output. Isi Topic kampus/labiot/telemetri, ubah Output menjadi a parsed JSON object, lalu klik Done.



4. Menambahkan node debug dari kategori common, lalu hubungkan keluaran mqtt in ke masukan debug dengan menarik wire.



5. Menekan tombol Deploy. Pastikan indikator di bawah mqtt in berubah menjadi hijau bertuliskan connected. Screenshot.
6. Mengamati data di Debug Sidebar (ikon kumbang di sisi kanan). Pastikan objek JSON masuk setiap 4 detik, berisi node_id, suhu, kelembapan, dan status_led.

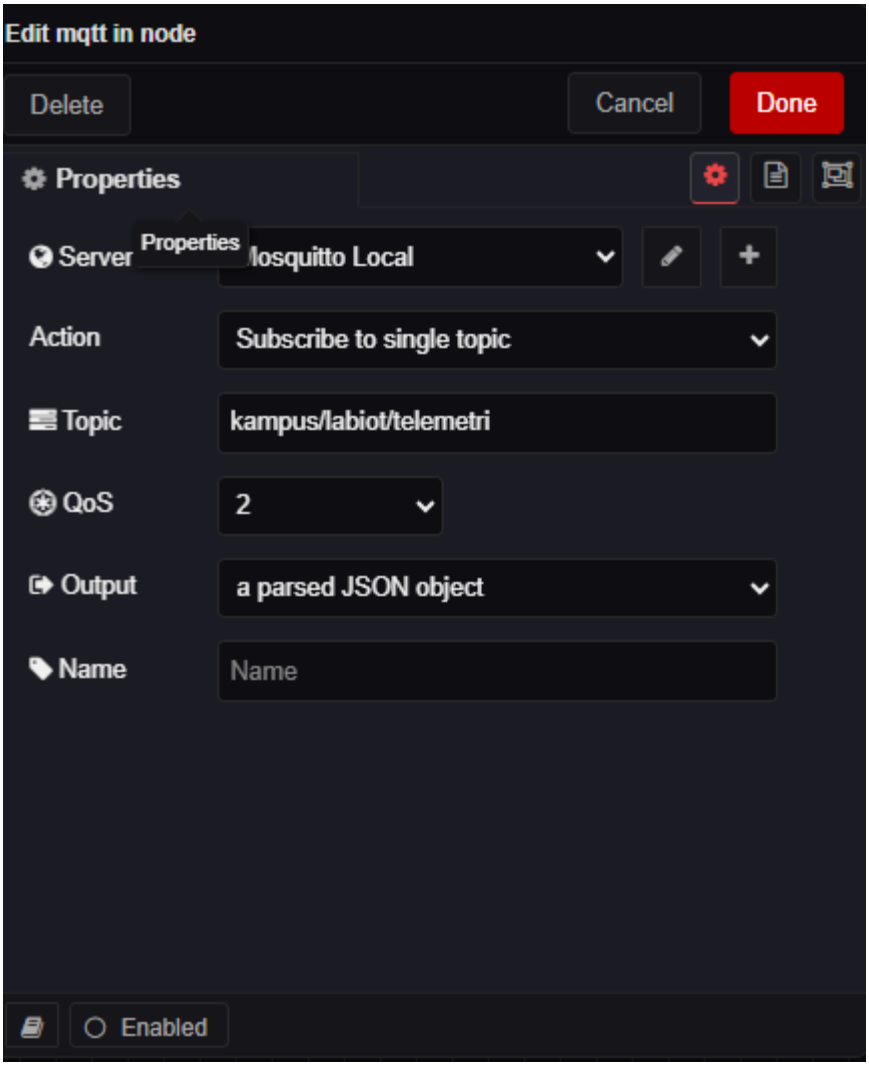


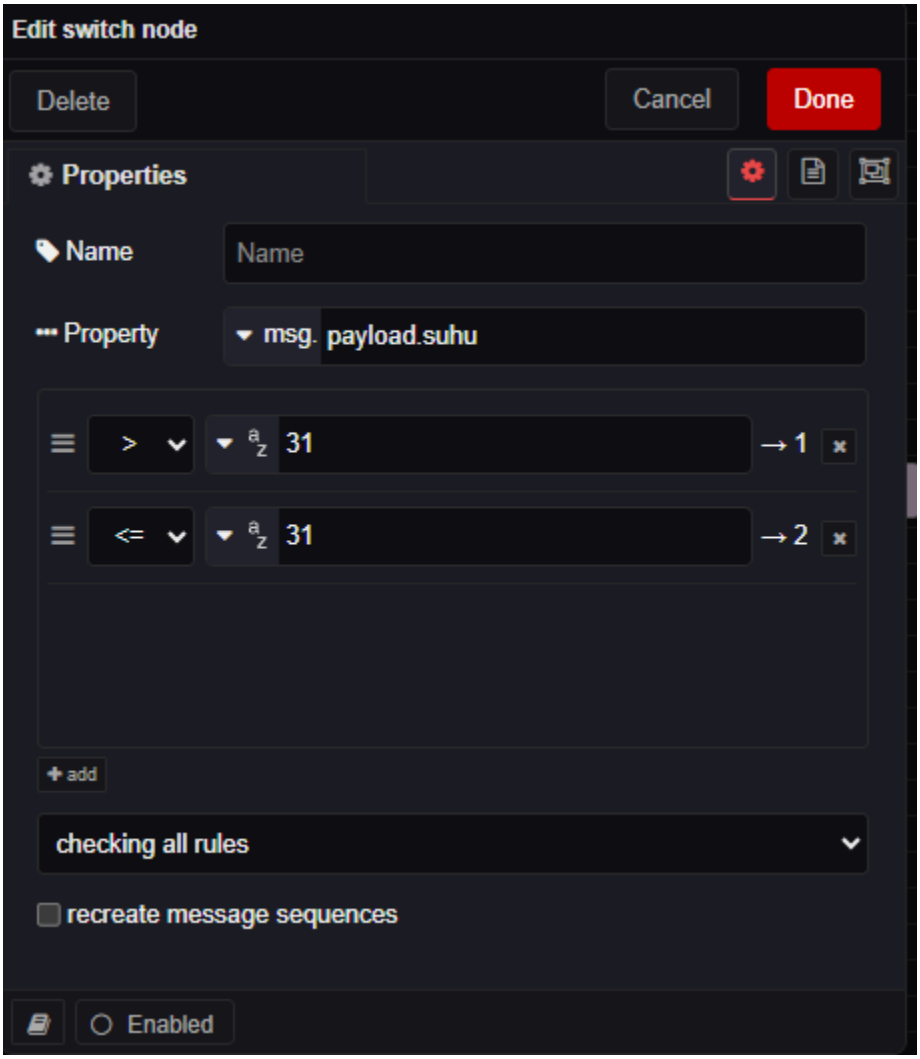
7.5.4

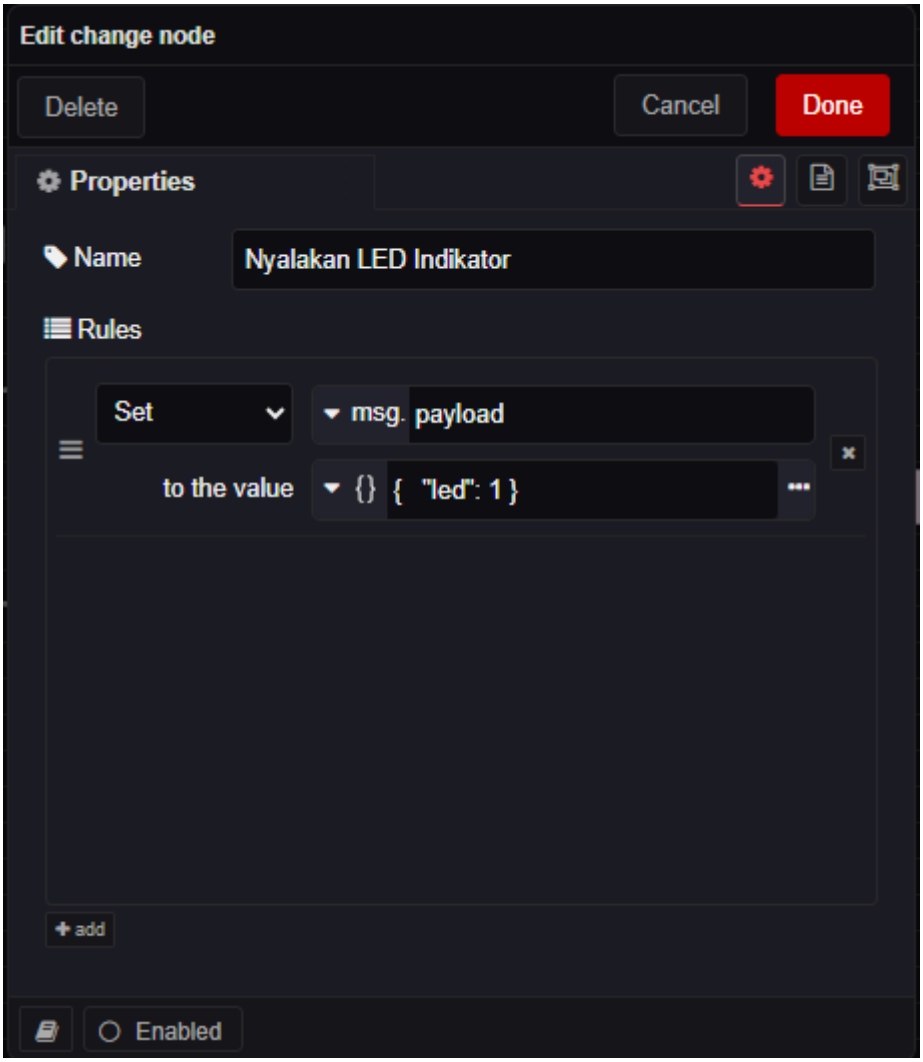
Langkah Kerja

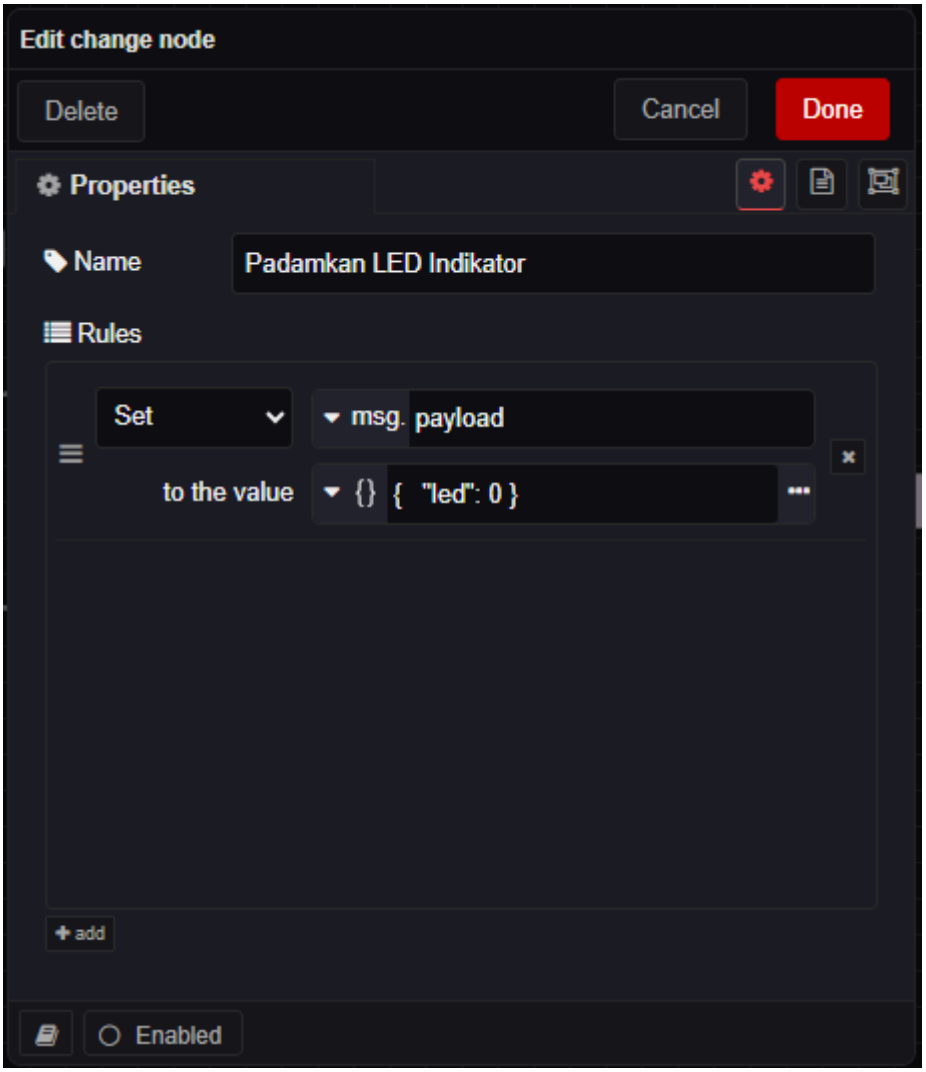
1. Menambahkan node switch dari kategori function, lalu hubungkan keluaran mqtt in ke masukan switch.
2. Mengatur aturan switch. Property diganti menjadi msg.payload.suhu. Aturan 1: operator > dengan nilai 31. Klik + add, aturan 2: operator <= dengan nilai 31. Klik Done, maka node punya dua port keluaran.
3. Membuat node change pertama (kondisi panas). Hubungkan port keluaran 1 switch ke node ini. Name Nyalakan LED Indikator, Set msg.payload, To bertipe JSON dengan isi {"led": 1}.
4. Membuat node change kedua (kondisi normal). Hubungkan port keluaran 2 switch ke node ini. Name Padamkan LED Indikator, Set msg.payload, To bertipe JSON dengan isi {"led": 0}.
5. Menambahkan node mqtt out dari kategori network, lalu hubungkan keluaran kedua node change ke masukannya. Server Mosquitto Local, Topic kampus/labiot/kendali, QoS 0, Retain false.
6. Menekan Deploy. Pastikan indikator mqtt in dan mqtt out berstatus connected. Screenshot alur lengkap.
7. Mengamati kondisi awal. Saat suhu ruangan 31 °C atau kurang, LED padam.
8. Menguji kondisi panas. Tempelkan jari atau hembuskan napas hangat ke sensor DHT sampai suhu melewati 31 °C. LED menyala, dan Serial Monitor menampilkan [INSTRUKSI MASUK] dengan {"led":1} serta MENYALA (ON). Foto LED menyala dan

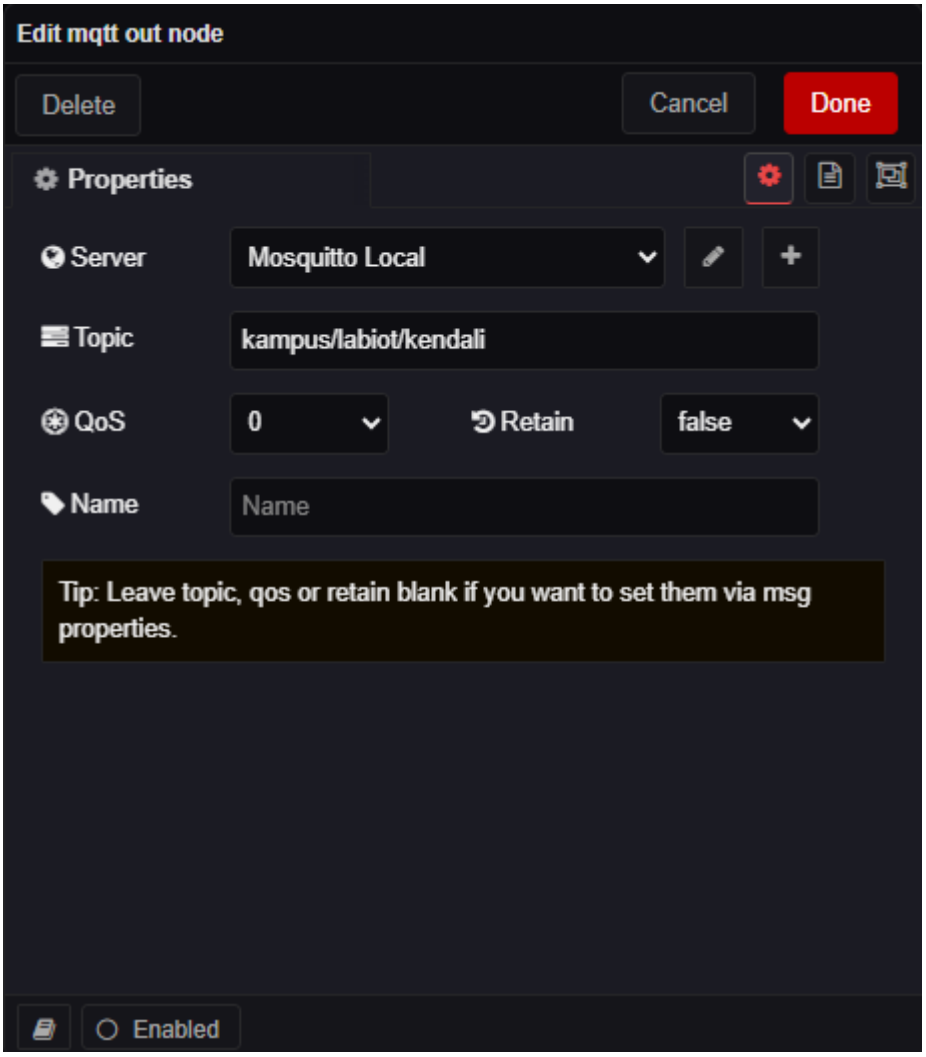
9. Menguji kembali ke normal. Lepaskan jari sampai suhu turun ke 31 °C atau kurang. LED padam otomatis.

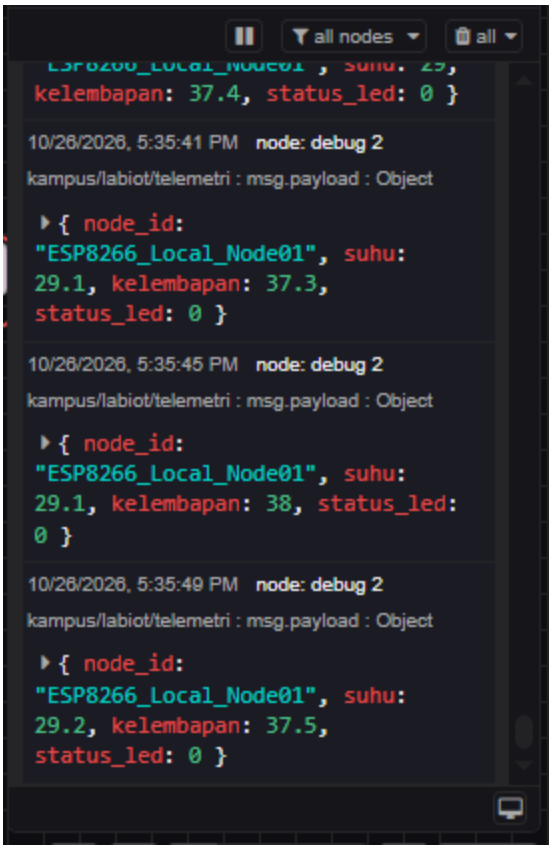


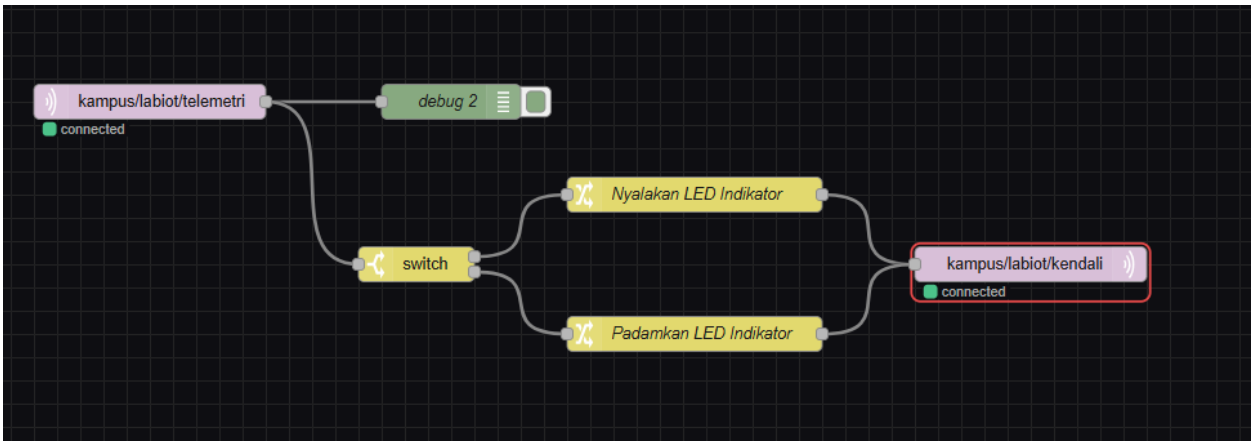

Edit switch node
Delete
Cancel
Done
Properties
Name
Name
Property
msg.
payload.suhu
>
31
→ 1
<=
31
→ 2
+ add
checking all rules
recreate message sequences
Enabled


Edit change node
Delete
Cancel
Done
Properties
Name
Nyalakan LED Indikator
Rules
Set
msg.
payload
to the value
{}
{ "led": 1 }
add
Enabled


Edit change node
Delete
Cancel
Done
Properties
Name
Padamkan LED Indikator
Rules
Set
msg.
payload
to the value
{}
{ "led": 0 }
+ add
Enabled


Edit mqtt out node
Delete
Cancel
Done
Properties
Server
Mosquitto Local
Topic
kampus/labiot/kendali
QoS
0
Retain
false
Name
Name
Tip: Leave topic, qos or retain blank if you want to set them via msg properties.
Enabled





Latihan 1: Jaringan Internal Docker dan Service Discovery

1. Kenapa localhost gagal?
Setiap kontainer punya jaringan sendiri yang terisolasi. Di dalam kontainer node-red, localhost menunjuk ke kontainer Node-RED itu sendiri, bukan ke laptop atau kontainer Mosquitto. Tidak ada program yang mendengarkan port 1883 di sana, sehingga koneksi ditolak (Connection Refused).

2. Bagaimana kontainer menemukan broker lewat nama layanan?
Docker Compose otomatis membuat jaringan bridge untuk proyek dan menyambungkan semua kontainer ke dalamnya. Di jaringan ini, Embedded DNS Server Docker mendaftarkan setiap nama layanan (mqtt, nodered). Saat Node-RED memanggil mqtt, DNS internal menjawab dengan IP kontainer Mosquitto, lalu koneksi berjalan langsung antar kontainer. IP boleh berubah setiap restart karena yang dipakai adalah nama.

Latihan 2: Konektivitas Jaringan dan Subnet

1. Kenapa NodeMCU tidak bisa publish?
NodeMCU (192.168.43.102) dan laptop (192.168.1.15) berada di subnet berbeda, sehingga domain broadcast-nya juga berbeda. Paket dari NodeMCU dikirim ke gateway hotspot, yang tidak punya rute ke jaringan laptop, jadi paket tidak pernah sampai ke broker.

2. Langkah teknis

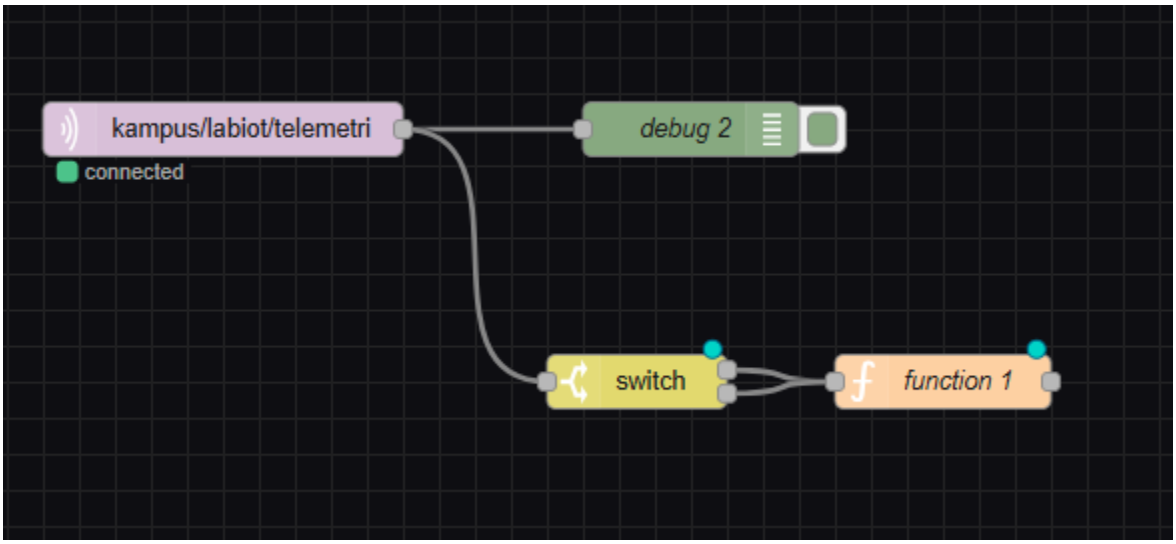
- Sambungkan laptop dan NodeMCU ke Wi-Fi atau hotspot yang sama, sehingga tiga angka awal IP-nya sama.
- Cek IP laptop dengan ipconfig, ubah mqttServer di kode ke IP itu, lalu upload ulang.
- Pastikan port 1883 diizinkan di firewall Windows.

Latihan 3: Filter Histeresis pada Rule Engine

Alur diubah menjadi mqtt in → function → mqtt out. Kode di Function Node:

```
let suhu = parseFloat(msg.payload.suhu);
let ledState = context.get("ledState") || 0;


if (suhu > 31.5 && ledState !== 1) {
    context.set("ledState", 1);
    msg.payload = { "led": 1 };
    return msg;
} else if (suhu < 30.0 && ledState !== 0) {
    context.set("ledState", 0);
    msg.payload = { "led": 0 };
  return msg;
}
return null;
```



Latihan 4: Volume Mount dan Persistensi Data

1. Kalau baris volume dihapus
Data Node-RED (flows.json dan pengaturan) hanya tersimpan di lapisan sementara milik kontainer. Setelah docker compose down, kontainer dan lapisan itu terhapus. Saat dinyalakan lagi, kanvas Node-RED kosong dan alur harus dibuat ulang. Data persistensi Mosquitto juga ikut hilang.

2. Bind Mount vs Named Volume
Bind Mount memetakan folder tertentu di laptop ke kontainer (contoh ./nodered:/data). Lokasinya kita tentukan sendiri dan filenya bisa langsung diedit dari host, tapi bergantung pada struktur folder host.
Named Volume dikelola Docker (contoh nodered_data:/data, dideklarasikan di bagian volumes:). Lebih portabel, tapi datanya tidak mudah diakses langsung dari host.
Perbedaan mendasarnya: lokasi bind mount ditentukan pengguna, lokasi named volume dikelola Docker.